Rapport d'Avancement

Projet Puissance 4 3D - MAM 3A

Maher Moujahed Moussa Chakroun Youssef Rais Mohamed Fadhel Bouzidi

Mai 2026

1 Introduction

Dans la continuité de notre projet de Génie Logiciel, après avoir modélisé l'infrastructure de base de notre Puissance 4 en 3D (le plateau de 20 bâtonnets 4x5x4, la gravité et la détection de victoire), notre équipe s'est concentrée sur le développement du moteur d'Intelligence Artificielle. Ce rapport présente nos dernières avancées majeures : la conception de fonctions d'évaluation heuristiques optimisées (rapide, moyenne, complète, avec détection de fourchettes) ainsi que l'amélioration des performances via le hachage de Zobrist et les tables de transposition.

2 Nouvelles Stratégies d'Évaluation de l'IA

Pour que notre IA puisse prendre des décisions pertinentes sans explorer l'intégralité de l'arbre des possibles, nous avons hiérarchisé nos fonctions d'évaluation en différentes classes, offrant des compromis variés entre précision et temps de calcul.

2.1 Évaluateur Moyen (Optimisation Binaire)

Afin de garantir des temps de réponse ultra-rapides, notamment pour le tri des coups (*Move Ordering*), nous avons implémenté la classe `EvaluatorMoyen`.

— **Masques Précalculés :** L'évaluation repose sur 122 masques binaires (représentant toutes les lignes de 4 cases possibles) couplés à l'instruction processeur `__builtin_popcountll` pour compter les perles en seulement deux cycles CPU.

— **Performance :** Cette méthode permet d'identifier les alignements partiels avec une exécution 27 fois plus rapide qu'une vérification classique (environ 0.4 µs par appel).

2.2 Évaluateur Avancé et Détection de Fourchettes

Utilisé spécifiquement sur les feuilles de notre algorithme Alpha-Beta, l'`EvaluatorAvance` analyse l'espace tridimensionnel avec une granularité fine.

— **Analyse Multidirectionnelle :** Il évalue les 13 directions de l'espace pour repérer les fenêtres non bloquées et attribue des scores progressifs (ex : +100 pour 3 perles alignées libres).

— **La Fourchette :** L'avancée stratégique majeure est la fonction `scoreFourchettes`. Elle détecte les cases vides où convergent au moins deux lignes de menaces de notre couleur. L'algorithme attribue un bonus massif (+500 pour une double menace, +1500 pour une triple) car dans un environnement 3D, ces configurations sont mathématiquement impossibles à bloquer en un seul tour par l'adversaire.

3 Optimisation : Hachage de Zobrist et Tables de Transposition

Dans le Puissance 4, un même état de la grille `Cell grille[4][5][4]` peut être atteint via différentes séquences de coups (transpositions). Pour éviter de recalculer inutilement l'évaluation d'un plateau déjà visité, nous avons mis en place une architecture de mémorisation.

3.1 Hachage de Zobrist

Le hachage de Zobrist nous permet d'associer un entier unique sur 64 bits à chaque configuration du plateau. À chaque coup joué ou annulé, la clé de la position est mise à jour instantanément via une simple opération binaire `XOR`. Cette complexité en $O(1)$ rend l'identification du plateau extrêmement légère pour le processeur.

3.2 Tables de Transposition

Cette clé de Zobrist sert d'index dans notre Table de Transposition. Lorsqu'une position est évaluée, son score et sa profondeur de recherche y sont sauvegardés. Avant d'explorer un nouveau nœud, l'IA consulte la table : si la position y figure déjà avec une profondeur suffisante, l'arbre est élagué et le score est renvoyé directement. Cela a considérablement réduit notre facteur de branchement, nous permettant de "voir" plus loin dans le jeu.

4 Conclusion et Prochaines Étapes

Le passage d'un simple stockage de données à une véritable intelligence artificielle capable d'anticipation tridimensionnelle rapide est désormais opérationnel. La prochaine étape décisive consistera à intégrer le nouvel environnement d'arbitrage fourni récemment pour le projet. Nous allons devoir incorporer la classe `joueur` à notre application afin de la faire communiquer avec le programme `arbitre` via MPI (Message Passing Interface). L'objectif du prochain sprint sera de traduire nos formats de mouvements internes vers la classe `position` demandée par l'arbitre, afin de pouvoir lancer nos exécutables en parallèle (`exe1` et `exe2`) et d'entraîner notre IA contre elle-même ou d'autres équipes dans les conditions réelles du tournoi final.